C-DAC Mumbai

MySQL Stored Functions

1. What is a Stored Function?

A stored function is a named program stored inside MySQL. It can accept input, perform processing, and must return one value.

Easy idea: Input → Function → One returned value.

```
get_grade(85)
      ↓
     'A'
```

Use a function when the main requirement is to calculate or return a value that can be used as part of an SQL expression.

2. Function vs Procedure

Feature	Function	Procedure
Main purpose	Returns a value	Performs a task
Return value	Must return one value	Does not have to return a value
RETURNS	Required	Not used
RETURN	Used to return the function value	Not used for returning a function value
Calling	Usually SELECT function_name(...)	CALL procedure_name(...)
SELECT / WHERE / ORDER BY	Can be used	Not directly
Parameter modes	Input parameters	IN, OUT, INOUT

Remember: Function → gives a value. Procedure → performs a task.

SHOW VARIABLES LIKE 'log_bin_trust_function_creators';

SET GLOBAL log_bin_trust_function_creators = 1;

3. Basic Function Syntax

```
DELIMITER //

CREATE FUNCTION function_name(parameter)
RETURNS data_type
BEGIN
    -- statements
    RETURN value;
END //
```

```
DELIMITER ;
```

RETURNS specifies the data type of the returned value. RETURN supplies the actual value.

4. Function Without a Parameter

```
delimiter //


Create function get_comapany_name()

Returns varchar(50)

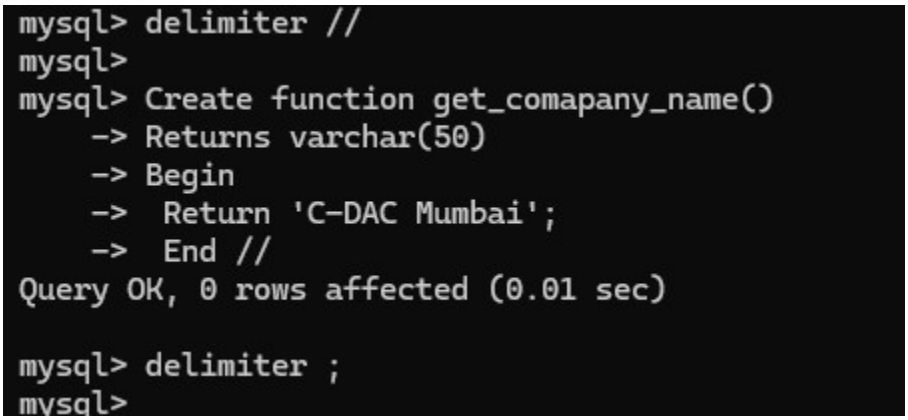
Begin

 Return 'C-DAC Mumbai';

 End //


 Demiliter;


 select get_comapany_name();
```



5. Function With One Parameter

DETERMINISTIC, NO SQL, or READS SQL DATA

2. Function with one parameter;

```
delimiter //

create function square_number( N int)

returns int

Deterministic

No sql

reads sql data

Begin

return N*N;



end //

delimiter ;
```

```
mysql> delimiter //
mysql> create function square_number( N int)
    -> returns int
    -> Begin
    ->
    -> return N*N;
    ->
    -> end//
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> select square_number(5);
+------------------+
| square_number(5) |
+------------------+
|               25 |
+------------------+
1 row in set (0.00 sec)

mysql> select square_number(10);
+-------------------+
| square_number(10) |
+-------------------+
|               100 |
+-------------------+
```

6. Function With Multiple Parameters

```
delimiter //

create function total_amount( price decimal(10,2), quantity int)

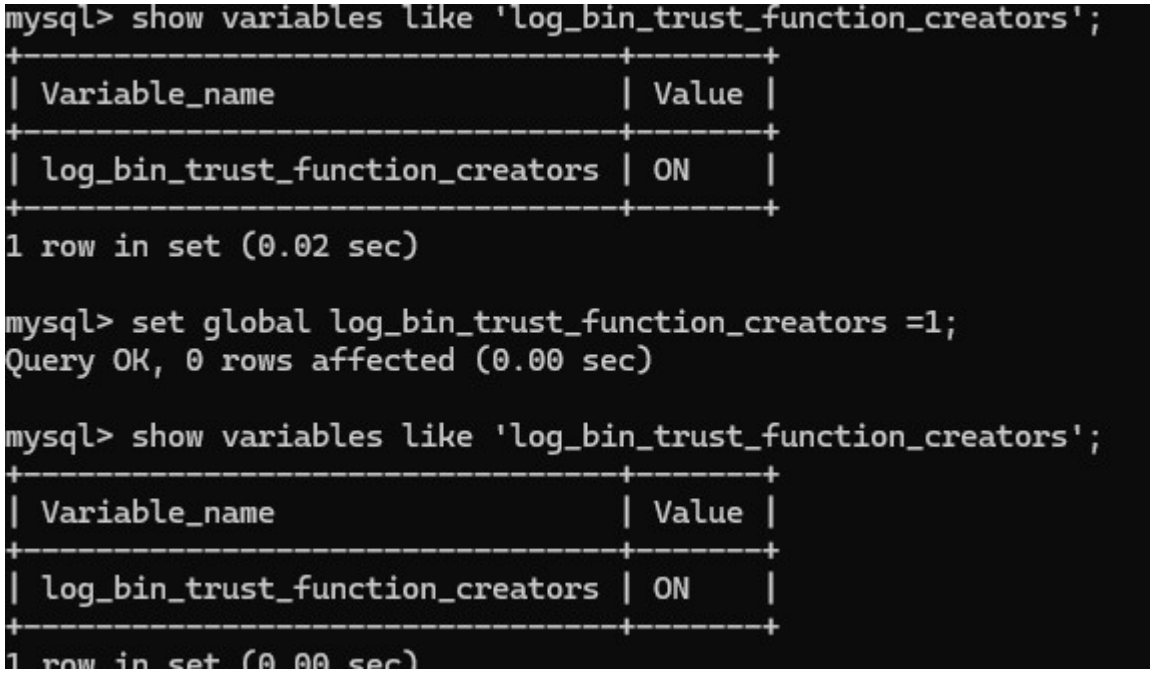
returns decimal (12,2)

begin

return price * quantity;


end //

delimiter ;
```


mysql> show variables like 'log_bin_trust_function_creators';
| Variable_name | Value |
| log_bin_trust_function_creators | ON |
1 row in set (0.02 sec)
mysql> set global log_bin_trust_function_creators =1;
Query OK, 0 rows affected (0.00 sec)
mysql> show variables like 'log_bin_trust_function_creators';
| Variable_name | Value |
| log_bin_trust_function_creators | ON |

```
mysql> create function total_amount( price decimal(10,2), quantity int)
    ->
    -> returns decimal (12,2)
    -> begin
    ->
    -> return price * quantity;
    ->
    -> end //
Query OK, 0 rows affected (0.00 sec)

mysql> delimiter ;
mysql>
mysql> select total_amount(56.20, 10);
+------------------------+
| total_amount(56.20, 10) |
+------------------------+
|                 562.00 |
+------------------------+
1 row in set (0.00 sec)

mysql> select total_amount(5623232322111.20, 10);
ERROR 1264 (22003): Out of range value for column 'price' at row 1
mysql> select total_amount(5623.20, 10);
+--------------------------+
| total_amount(5623.20, 10) |
+--------------------------+
|                56232.00 |
+--------------------------+
1 row in set (0.00 sec)
```

— Student Grade

Use this as the main classroom example because it demonstrates parameters, IF/ELSEIF, RETURN, SELECT, WHERE and ORDER BY.

```
delimiter //

create function get_grade1(s_marks int)

Returns varchar(2)

begin

if s_marks >= 90 then

return 'A+';

elseif s_marks >= 80 then

return 'A';
```

```
elseif s_marks >= 70 then

return 'B';

elseif s_marks >= 60 then

return 'C';

elseif s_marks >= 50 then

return 'D';

else return 'F';

end if;

end //

delimiter ;
```

```
mysql> create function get_grade1(s_marks int)
    -> Returns varchar(2)
    ->
    -> begin
    ->
    -> if s_marks >= 90 then
    -> return 'A+';
    ->
    -> elseif s_marks >= 80 then
    -> return 'A';
    ->
    -> elseif s_marks >= 70 then
    -> return 'B';
    ->
    -> elseif s_marks >= 60 then
    -> return 'C';
    ->
    -> elseif s_marks >= 50 then
    -> return 'D';
    ->
    -> else return 'F';
    ->
    -> end if;
    ->
    -> end //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> delimiter ;
mysql>
mysql> select get_grade1(95);
+----------------+
| get_grade1(95) |
+----------------+
```

8. Function in SELECT

```
SELECT name,
       marks,
       get_grade(marks) AS grade
FROM student;
```

The function is evaluated for each row, using that row's marks as the input.

```
mysql> select student_id,name, marks, get_grade1(marks) from student;
+------------+-----------------+-------+--------------------+
| student_id | name            | marks | get_grade1(marks) |
+------------+-----------------+-------+--------------------+
|        101 |    Amit Patil   | 85.50 | A                  |
|        102 | Neha chaudhari  | 92.00 | A+                 |
|        103 | Rahul khot      | 76.50 | B                  |
|        104 | Priya bapat     | 88.00 | A                  |
|        106 | Akash yadav     | 30.00 | F                  |
+------------+-----------------+-------+--------------------+
5 rows in set (0.00 sec)
```

9. Function in WHERE

```
SELECT name, marks
FROM student
WHERE get_grade1(marks) = 'A';
```

The function calculates a grade and WHERE filters the result.

```
mysql> select student_id, name, marks from student where get_grade1(marks) = 'A';
+------------+-----------------+-------+
| student_id | name            | marks |
+------------+-----------------+-------+
|        101 |    Amit Patil   | 85.50 |
|        104 | Priya bapat     | 88.00 |
+------------+-----------------+-------+
2 rows in set (0.00 sec)

mysql> select student_id, name, marks, get_grade1(marks) as grade from student order by grade;
+------------+-----------------+-------+-------+
| student_id | name            | marks | grade |
+------------+-----------------+-------+-------+
|        101 |    Amit Patil   | 85.50 | A     |
|        104 | Priya bapat     | 88.00 | A     |
|        102 | Neha chaudhari  | 92.00 | A+    |
|        103 | Rahul khot      | 76.50 | B     |
```

10. Function in ORDER BY

```
SELECT name,
       marks,
       get_grade1(marks) AS grade
FROM student order by grade;


SELECT name, marks,get_grade1(marks) AS grade FROM student order by grade
```

A function can be used in ORDER BY. This sorts the text values alphabetically. For academic ranking, ORDER BY marks DESC is usually more appropriate.

11. Practical Example — Annual Salary

```
delimiter //

create function annual_package(Monthly_salary decimal (10,2))

returns decimal (12,2)

begin

return (Monthly_salary + (Monthly_salary * 0.30))*12;

end //

delimiter ;
```

```
mysql> delimiter //
mysql>
mysql> create function annual_package(Monthly_salary decimal (10,2))
    -> returns decimal (12,2)
    ->
    -> begin
    ->
    -> return (Monthly_salary + (Monthly_salary * 0.30))*12;
    ->
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
```

```
mysql> select annual_package(60000);
+-----------------------+
| annual_package(60000) |
+-----------------------+
|             936000.00 |
+-----------------------+
1 row in set (0.00 sec)

mysql> select * from employee;
+--------+----------+----------+
| emp_id | emp_name | salary   |
+--------+----------+----------+
|    101 | Amit     | 80000.00 |
|    102 | Priya    | 50000.00 |
|    103 | Rahul    | 45000.00 |
+--------+----------+----------+
3 rows in set (0.00 sec)

mysql> select emp_id, emp_name, salary, annual_package(salary) as annual_package from employee;
+--------+----------+----------+----------------+
| emp_id | emp_name | salary   | annual_package |
+--------+----------+----------+----------------+
|    101 | Amit     | 80000.00 |     1248000.00 |
|    102 | Priya    | 50000.00 |      780000.00 |
|    103 | Rahul    | 45000.00 |      702000.00 |
+--------+----------+----------+----------------+
3 rows in set (0.00 sec)

mysql> show function status where db='oct14';
```

12. Practical Example — Discount

```
delimiter //

create function calculate_discount( amount decimal(10,2))

Returns decimal (10,2)

Begin

 if amount >= 20000 then

 return amount * 0.10;




 elseif amount >= 10000 then

 return amount * 0.20;

 else

  return amount * 0.03;

  end if;

 end //

  delimiter ;
```

```
mysql> delimiter //
mysql>
mysql> create function calculate_discount( amount decimal(10,2))
    -> Returns decimal (10,2)
    -> Begin
    ->  if amount >= 10000 then
    ->  return amount * 0.10;
    ->
    ->  elseif amount >= 20000 then
    ->  return amount * 0.20;
    ->
    ->  else
    ->   return amount * 0.03;
    ->
    ->   end if;
    ->
    ->   end //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> delimiter ;
mysql>
mysql> select calculate_discount(12000);
+--------------------------+
| calculate_discount(12000) |
+--------------------------+
|                  1200.00 |
+--------------------------+
```

13. RETURNS vs RETURN

Keyword	Purpose	Example
RETURNS	Specifies return data type	RETURNS INT
RETURN	Supplies actual returned value	RETURN a + b

14. Function Characteristics

For a short course, focus on these three characteristics.

Characteristic	Meaning	Example
DETERMINISTIC	Same input produces the same result.	square_number(5) always returns 25.
NO SQL	The function does not access SQL data.	A function that only calculates a + b.
READS SQL DATA	The function reads data from database tables.	A function that counts employees.

15. DETERMINISTIC + NO SQL Example

```
DELIMITER //

CREATE FUNCTION square_number2(n INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    RETURN n * n;
END //

DELIMITER ;
```

The same input always gives the same result, and the function does not read or modify table data.

17. log_bin_trust_function_creators

Depending on binary logging and server configuration, MySQL may produce an error when creating stored functions. Check the setting:

```
SHOW VARIABLES LIKE 'log_bin_trust_function_creators';
```

For a local classroom server, an administrator may temporarily use:

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

This is a server configuration setting, not normal function syntax. Do not change it blindly on production or replicated servers.

19. Calling and Managing Functions

```
SELECT get_grade(85);
SHOW CREATE FUNCTION get_grade;
DROP FUNCTION get_grade;
```

SHOW CREATE FUNCTION displays the stored definition. DROP FUNCTION removes the function; it does not remove table data.

20. Important Points to Remember

- A stored function must return one value.
- RETURNS specifies the return data type.
- RETURN supplies the actual returned value.
- A function can have zero, one, or multiple input parameters.
- Functions can be used in SELECT, WHERE and ORDER BY.
- DETERMINISTIC means the same input produces the same result.
- NO SQL means the function does not access SQL data.
- READS SQL DATA means the function reads database data.
- Function parameters are input parameters; OUT and INOUT are procedure parameter modes.
- Use SHOW CREATE FUNCTION to inspect a function and DROP FUNCTION to remove it.

22. Quick Student Practice

1. Create a function that converts Celsius to Fahrenheit.
2. Create a function that returns PASS if marks >= 50, otherwise FAIL.
3. Create a function that calculates 10% tax on an amount.
4. Use a function in SELECT to display annual salary.
5. Use a function in WHERE to filter rows using a calculated value.

1. Why Do We Need a Trigger?

Suppose we have an employee table and want to maintain a history of salary changes. Whenever a salary is updated, we want the old salary, new salary, employee ID, and change time to be recorded automatically.

```
UPDATE employee
SET salary = 65000
WHERE emp_id = 101;
```

Without a trigger, the application would have to remember to execute a second INSERT into an audit table. A trigger allows the database to perform that action automatically.

Definition: A trigger is a database program that automatically executes when a specified event occurs on a table.

Simple idea: Event happens → Trigger automatically fires → Trigger executes its logic.

2. Trigger vs Stored Procedure

Stored Procedure	Trigger
Executed explicitly	Executed automatically
Called using CALL	No CALL is required
Developer decides when to execute it	The table event causes it to execute
Used for operations/business logic	Used for automatic actions related to table events

```
-- Procedure
CALL calculate_bonus(101);

-- Trigger: no CALL is written
UPDATE employee
SET salary = 65000
WHERE emp_id = 101;
```

Easy way to remember:

- Procedure = “Do this when I call you.”
- Trigger = “Do this automatically when this event happens.”

3. Trigger Timing and Events

A trigger is defined using a timing and an event.

Timing	Event	Meaning
BEFORE	INSERT	Before a new row is inserted
AFTER	INSERT	After a new row is inserted
BEFORE	UPDATE	Before an existing row is updated

AFTER	UPDATE	After an existing row is updated
BEFORE	DELETE	Before an existing row is deleted
AFTER	DELETE	After an existing row is deleted

BEFORE means the trigger runs before the requested table operation. It is useful for validation or changing incoming values.

AFTER means the trigger runs after the requested operation. It is useful for audit entries, updating related information, or maintaining summaries.

4. Row-Level Triggers

MySQL triggers are row-level triggers. The trigger definition contains FOR EACH ROW.

```
CREATE TRIGGER trigger_name
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
    -- trigger logic
END;
```

If an UPDATE changes 10 employees, the row-level trigger can execute 10 times—once for each affected row. MySQL does not support statement-level triggers.

5. OLD and NEW – Most Important Concept

OLD and NEW are special references to row values inside a trigger.

Event	**OLD**	**NEW**
INSERT	Not available	Available
UPDATE	Available	Available
DELETE	Available	Not available

Remember:

- INSERT → NEW
- UPDATE → OLD + NEW
- DELETE → OLD

OLD represents the value before the operation. NEW represents the new value associated with the operation. For UPDATE, both versions are available.

6. CREATE TRIGGER – General Syntax

```
DELIMITER //

CREATE TRIGGER trigger_name
BEFORE | AFTER
INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- trigger statements
END //
```

```
DELIMITER ;
```

A trigger belongs to a table and is associated with one timing/event combination.

7. Main Practical Example – Salary Audit

This is the central practical example. Whenever an employee's salary changes, the database automatically records the old and new salary.

Create audit table

```
create table  audit_table(
   audit_id int auto_increment primary key,
   emp_id int,
   emp_name varchar(30),
   old_salary decimal(10,2),
   new_salary decimal (10,2),
   change_on datetime);
```

Create the trigger

```
delimiter //

   create trigger trg_salary_audit
   after update on employee
   for each row
   begin
   insert into audit_table (emp_id, emp_name, old_salary, new_salary, change_on)
   values
   (new.emp_id, new.emp_name, old.salary, new.salary, now());

   end //

   delimiter ;
```

```
mysql> create table  audit_table(
    ->    audit_id int auto_increment primary key,
    ->    emp_id int,
    ->    emp_name varchar(30),
    ->    old_salary decimal(10,2),
    ->    new_salary decimal (10,2),
    ->    change_on datetime);
Query OK, 0 rows affected (0.02 sec)

mysql> desc audit_table;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| audit_id   | int          | NO   | PRI | NULL    | auto_increment |
| emp_id     | int          | YES  |     | NULL    |                |
| emp_name   | varchar(30)  | YES  |     | NULL    |                |
| old_salary | decimal(10,2)| YES  |     | NULL    |                |
| new_salary | decimal(10,2)| YES  |     | NULL    |                |
| change_on  | datetime     | YES  |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

```
mysql> delimiter //
mysql>
mysql>    create trigger trg_salary_audit
    ->    after update on employee
    ->    for each row
    ->    begin
    ->    insert into audit_table (emp_id, emp_name, old_salary, new_salary, change_on)
    ->    values
    ->    (new.emp_id, new.emp_name, old.salary, new.salary, now());
    ->
    ->    end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> update employee set salary=90000 where emp_id=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from employee;
+--------+----------+----------+
| emp_id | emp_name | salary   |
+--------+----------+----------+
|    101 | Amit     | 90000.00 |
|    102 | Priya    | 50000.00 |
|    103 | Rahul    | 45000.00 |
+--------+----------+----------+
3 rows in set (0.00 sec)

mysql> select * from audit_table;
+----------+--------+----------+------------+------------+---------------------+
| audit_id | emp_id | emp_name | old_salary | new_salary | change_on           |
+----------+--------+----------+------------+------------+---------------------+
|        1 |    101 | Amit     |   80000.00 |   90000.00 | 2026-10-16 12:06:40 |
+----------+--------+----------+------------+------------+---------------------+
```

```
mysql> update employee set salary=70000 where emp_id=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from employee;
+--------+----------+----------+
| emp_id | emp_name | salary   |
+--------+----------+----------+
|    101 | Amit     | 70000.00 |
|    102 | Priya    | 50000.00 |
|    103 | Rahul    | 45000.00 |
+--------+----------+----------+
3 rows in set (0.00 sec)

mysql> select * from audit_table;
+----------+--------+----------+------------+------------+---------------------+
| audit_id | emp_id | emp_name | old_salary | new_salary | change_on           |
+----------+--------+----------+------------+------------+---------------------+
|        1 |    101 | Amit     |   80000.00 |   90000.00 | 2026-10-16 12:06:40 |
|        2 |    101 | Amit     |   90000.00 |   70000.00 | 2026-10-16 12:08:08 |
+----------+--------+----------+------------+------------+---------------------+
2 rows in set (0.00 sec)

mysql> update employee set emp_name='Amit Patil' where emp_id=101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

8. Understanding OLD and NEW in the Audit Example

```
OLD.salary
NEW.salary
```

- OLD.salary = salary before the UPDATE
- NEW.salary = salary after the UPDATE

If the salary changes from 60000 to 70000, then OLD.salary = 60000 and NEW.salary = 70000.

9. BEFORE INSERT – Validation Example

Use a BEFORE trigger when incoming data should be checked or changed before it is stored. For example, prevent a negative salary.

```
DELIMITER //

CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //

DELIMITER ;
```

```
INSERT INTO employee
VALUES (104, 'Sneha', -5000);
```

The trigger runs before the row is inserted and rejects the invalid value.

10. BEFORE vs AFTER – Practical Rule

Use BEFORE when...	Use AFTER when...
Validating incoming data	Creating an audit record
Preventing an invalid operation	Updating related/summary information
Changing incoming values	Performing an action after the original operation succeeds

Simple rule: BEFORE is commonly used for validation or modification before the operation. AFTER is commonly used for audit and related actions after the original operation succeeds.

11. The Six Trigger Combinations

Trigger	Typical use	OLD/NEW
BEFORE INSERT	Validation or modify incoming data	NEW
AFTER INSERT	Audit or related action	NEW
BEFORE UPDATE	Validate/prevent a change	OLD + NEW
AFTER UPDATE	Audit/history	OLD + NEW
BEFORE DELETE	Validate/prevent deletion	OLD
AFTER DELETE	Archive/audit deleted data	OLD

12. Example – Auto-Updating Stock

Suppose an order is inserted. We want product stock to decrease automatically.

```
create table product ( product_id int primary key,

             product_name varchar(30),  int );

create table orders ( order_id int, product_id int, quantity int);
```

```
delimiter //

create trigger trg_stock_update

after insert on orders

for each row

begin

update product

set stock= stock - new.quantity

where product_id= new.product_id;
```

```
end //

delimiter ;
```

```
mysql> insert into product values (1, 'Iphone_17', 300), (2, 'SamsungPro', 250), (3, 'Nokia', 500);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> delimiter //
mysql>
mysql> create trigger trg_stock_update
    -> after insert on orders
    -> for each row
    ->
    -> begin
    -> update product
    -> set stock= stock - new.quantity
    -> where product_id= new.product_id;
    -> end //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
mysql>
mysql> select * from product;
+------------+--------------+-------+
| product_id | product_name | stock |
+------------+--------------+-------+
|          1 | Iphone_17    |   300 |
```

```
mysql> insert into orders values (1, 1, 50);
Query OK, 1 row affected (0.01 sec)

mysql> select * from orders;
+----------+------------+----------+
| order_id | product_id | quantity |
+----------+------------+----------+
|        1 |          1 |       50 |
+----------+------------+----------+
1 row in set (0.00 sec)

mysql> select * from product;
+------------+--------------+-------+
| product_id | product_name | stock |
+------------+--------------+-------+
|          1 | Iphone_17    |   250 |
|          2 | SamsungPro   |   250 |
|          3 | Nokia        |   500 |
+------------+--------------+-------+
3 rows in set (0.00 sec)

mysql> insert into orders values (1, 2, 100);
Query OK, 1 row affected (0.01 sec)

mysql> select * from product;
+------------+--------------+-------+
| product_id | product_name | stock |
+------------+--------------+-------+
|          1 | Iphone_17    |   250 |
|          2 | SamsungPro   |   150 |
|          3 | Nokia        |   500 |
+------------+--------------+-------+
3 rows in set (0.00 sec)
```

13. Multiple Triggers on the Same Event

MySQL allows multiple triggers for the same timing/event combination. Execution order can be controlled using FOLLOWS or PRECEDES.

```
CREATE TRIGGER trg_second
AFTER UPDATE ON employee
FOR EACH ROW
FOLLOWS trg_first
BEGIN
    -- logic
END;
```

FOLLOWS means the new trigger executes after the named trigger. PRECEDES means it executes before the named trigger.

For a beginner class, understand the concept and syntax, but do not spend too much time on multiple-trigger ordering.

15. DROP TRIGGER

```
DROP TRIGGER trg_salary_audit;
```

DROP TRIGGER removes the trigger definition. It does not remove the table.

16. SHOW TRIGGERS

```
SHOW TRIGGERS;
SHOW TRIGGERS LIKE 'employee';
```

Use SHOW TRIGGERS during practical demonstrations to verify that triggers exist.

17. Important Trigger Limitations

17.1 No COMMIT or ROLLBACK inside a trigger

A trigger cannot explicitly perform transaction control such as COMMIT or ROLLBACK. The trigger executes as part of the statement that fired it.

17.2 No result sets

A trigger is intended to perform an automatic database action. It cannot behave like a normal query that returns a result set to the client.

17.3 TRUNCATE does not fire DELETE triggers

```
DELETE FROM employee;
-- DELETE triggers can fire

TRUNCATE TABLE employee;
-- DELETE triggers do not fire
```

Remember: TRUNCATE does not fire DELETE triggers.

18. DELETE vs TRUNCATE – Trigger Point of View

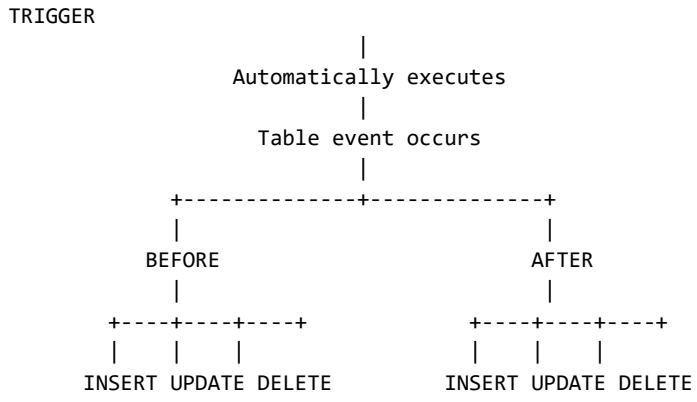
DELETE	TRUNCATE
DML operation	DDL-like operation in MySQL
Can use WHERE	Cannot use WHERE
DELETE triggers can fire	DELETE triggers do not fire
Rows are removed through DELETE	Table is emptied by truncation

19. Quick Revision – What Students Must Remember

- A trigger executes automatically when a specified table event occurs.
- A procedure is explicitly called; a trigger is automatically fired.
- BEFORE/AFTER tells when the trigger runs.
- INSERT/UPDATE/DELETE tells which event fires it.

- MySQL triggers are row-level and use FOR EACH ROW.
- INSERT → NEW
- UPDATE → OLD + NEW
- DELETE → OLD
- BEFORE is commonly used for validation or modifying incoming data.
- AFTER is commonly used for audit and related actions.
- TRUNCATE does not fire DELETE triggers.
- A trigger cannot contain explicit COMMIT/ROLLBACK.

21. One Board Diagram for the Class



22. Golden Rule

Use a trigger when the database should automatically perform an action whenever a specific table event occurs.